

Apêndice C

Sistemas de Numeração

Exercícios — Resolução Didática

“Resolvemos os 11 exercícios C.16 a C.26. Eles cobrem o mesmo terreno da autorrevisão — conversões entre bases e representação de negativos — mas com novos valores e duas curiosidades: a base 12 (C.16) e as bases 6, 13 e 3 (C.17). Cada resposta inclui verificação matemática cruzada.”

1 Exercício C.16 — Sistema base 12

Enunciado. A base 12 seria mais fácil porque 12 é divisível por mais números que 10. Qual o menor dígito da base 12? Qual o maior? Quais os valores posicionais das 4 posições da direita?

Menor dígito

0 (zero), assim como em *qualquer* sistema posicional.

Maior dígito

Um a menos que a base, ou seja, 11.

Como não temos um único símbolo decimal para 10 e 11, precisamos inventar novos símbolos. Convenções históricas e duozimais propuseram:

Valor	Símbolo do livro Deitel	Símbolos duozimais históricos
10	A	X (Pitman) ou T (Bell)
11	B	E (Pitman) ou L (Bell)

Adotamos as letras A e B no estilo hexadecimal.

Valores posicionais das 4 posições da direita

São potências de 12:

$$12^0 = \boxed{1}, \quad 12^1 = \boxed{12}, \quad 12^2 = \boxed{144}, \quad 12^3 = \boxed{1728}$$

P3	P2	P1	P0
1728	144	12	1

Observação

Por que duodécimo seria mais conveniente? 12 tem divisores 1, 2, 3, 4, 6, 12 (6 divisores), enquanto 10 tem apenas 1, 2, 5, 10 (4 divisores). Em base 12, a divisão por 3 e 4 dá resultado exato e simples (sem dízimas), o que ajudaria muito em cálculos cotidianos. O sistema sexagesimal (60), usado pelos sumérios para tempo e ângulos, leva a ideia ainda mais longe: 60 tem 12 divisores.

2 Exercício C.17 — Tabela de valores posicionais (bases 6, 13, 3)

Enunciado. Complete a tabela de valores posicionais.

Sistema	P3	P2	P1	P0
Decimal	1000	100	10	1
Base 6	216	36	6	1
Base 13	2197	169	13	1
Base 3	27	9	3	1

Cálculos

- **Base 6:** $6^0 = 1$, $6^1 = 6$, $6^2 = 36$, $6^3 = 216$.
- **Base 13:** $13^0 = 1$, $13^1 = 13$, $13^2 = 169$, $13^3 = 2197$.
- **Base 3:** $3^0 = 1$, $3^1 = 3$, $3^2 = 9$, $3^3 = 27$.

3 Exercício C.18 — Binário 100101111010 para octal e hex

Para octal: agrupe de 3 em 3 (a partir da direita)

$$\underbrace{100}_4 \underbrace{101}_5 \underbrace{111}_7 \underbrace{010}_2$$

Octal: 4572.

Para hexadecimal: agrupe de 4 em 4 (a partir da direita)

$$\underbrace{1001}_9 \underbrace{0111}_7 \underbrace{1010}_{A=10}$$

Hexadecimal: 97A.

Verificação

$$100101111010_2 = 2426_{10}.$$

$$\text{Conferindo octal: } 4572_8 = 4 \times 512 + 5 \times 64 + 7 \times 8 + 2 = 2048 + 320 + 56 + 2 = 2426.$$

$$\text{Conferindo hex: } 97A_{16} = 9 \times 256 + 7 \times 16 + 10 = 2304 + 112 + 10 = 2426.$$

4 Exercício C.19 — Hexadecimal 3A7D em binário

$$\begin{array}{cccc} \underbrace{3} & \underbrace{A} & \underbrace{7} & \underbrace{D} \\ 0011 & 1010 & 0111 & 1101 \end{array}$$

Binário: 0011 1010 0111 1101.

Verificação

$$3A7D_{16} = 3 \times 4096 + 10 \times 256 + 7 \times 16 + 13 = 12288 + 2560 + 112 + 13 = 14973_{10}.$$

5 Exercício C.20 — Hexadecimal 765F em octal

Dica do livro: converta primeiro para binário.

Etapas 1: hex 765F para binário

$$\begin{array}{cccc} \underbrace{7} & \underbrace{6} & \underbrace{5} & \underbrace{F} \\ 0111 & 0110 & 0101 & 1111 \end{array}$$

Binário: 0111 0110 0101 1111.

Etapas 2: binário para octal (agrupando de 3 em 3)

São 16 bits ao todo; precisamos completar com zeros à esquerda para totalizar um múltiplo de 3 (18 bits, 6 grupos):

$$\begin{array}{cccccc} \underbrace{000} & \underbrace{111} & \underbrace{011} & \underbrace{001} & \underbrace{011} & \underbrace{111} \\ 0 & 7 & 3 & 1 & 3 & 7 \end{array}$$

Octal: 073137 (ou simplesmente 73137).

Verificação

$$765F_{16} = 7 \times 4096 + 6 \times 256 + 5 \times 16 + 15 = 28672 + 1536 + 80 + 15 = 30303_{10}.$$

$$E\ 73137_8 = 7 \times 4096 + 3 \times 512 + 1 \times 64 + 3 \times 8 + 7 = 28672 + 1536 + 64 + 24 + 7 = 30303.$$

6 Exercício C.21 — Binário 1011110 em decimal

Posição	6	5	4	3	2	1	0
Valor pos.	64	32	16	8	4	2	1
Dígito	1	0	1	1	1	1	0
Produto	64	0	16	8	4	2	0

$$64 + 16 + 8 + 4 + 2 = \boxed{94_{10}}$$

Decimal: 94.

7 Exercício C.22 — Octal 426 em decimal

Posição	2	1	0
Valor pos.	64	8	1
Dígito	4	2	6
Produto	256	16	6

$$256 + 16 + 6 = \boxed{278_{10}}$$

Decimal: 278.

8 Exercício C.23 — Hexadecimal FFFF em decimal

Posição	3	2	1	0
Valor pos.	4096	256	16	1
Dígito	F (15)	F (15)	F (15)	F (15)
Produto	61440	3840	240	15

$$61440 + 3840 + 240 + 15 = \boxed{65535_{10}}$$

Decimal: 65535.

Observação

$65535 = 2^{16} - 1$ é o maior inteiro *sem sinal* representável em 16 bits. É o limite superior de `unsigned short` em muitas implementações. Também conhecido como “portas TCP/IP máximas” (65535 portas disponíveis). E é um divisor de Mersenne: $2^{16} - 1 = 3 \times 5 \times 17 \times 257$.

9 Exercício C.24 — Decimal 299 em bin, oct, hex

Binário

$$299 = 256 + 32 + 8 + 2 + 1:$$

Valor pos.	256	128	64	32	16	8	4	2
1								
Dígito	1	0	0	1	0	1	0	1
1								

Binário: 100101011.

Octal: agrupando o binário de 3 em 3

$$\underbrace{100}_4 \underbrace{101}_5 \underbrace{011}_3$$

Octal: 453.

Hexadecimal: agrupando o binário de 4 em 4

(Completamos com 3 zeros à esquerda para fechar 12 bits = 3 grupos.)

$$\underbrace{0001}_1 \underbrace{0010}_2 \underbrace{1011}_{B=11}$$

Hexadecimal: 12B.

Verificação

- Bin: $256 + 32 + 8 + 2 + 1 = 299$.
- Oct: $4 \times 64 + 5 \times 8 + 3 = 256 + 40 + 3 = 299$.
- Hex: $1 \times 256 + 2 \times 16 + 11 = 256 + 32 + 11 = 299$.

10 Exercício C.25 — Complemento de 1 e 2 de 779

Etapla 1: 779 em binário

$$779 = 512 + 256 + 8 + 2 + 1:$$

Valor pos.	512	256	128	64	32	16	8	4	2
1									
Dígito	1	1	0	0	0	0	1	0	1
1									

Binário: 1100001011 (10 bits).

Etapla 2: complemento de UM (inverte todos os bits)

$$\begin{array}{r|cccccccccc} 779 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ \hline \sim 779 \text{ (C1)} & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 \end{array}$$

Complemento de UM: 0011110100.

Etapla 3: complemento de DOIS (C1 + 1)

$$0011110100 + 1 = 0011110101$$

Complemento de DOIS: 0011110101.

Verificação

$$\begin{array}{r} 1100001011 \\ + 0011110101 \\ \hline (1) 0000000000 \end{array}$$

O bit transbordado fora dos 10 bits é descartado: resultado 0000000000 = 0.

11 Exercício C.26 — Complemento de dois de -1 em 32 bits

Enunciado. Mostre o complemento de dois do valor inteiro -1 em uma máquina com inteiros de 32 bits.

Etapla 1: +1 em binário de 32 bits

$$+1 = \underbrace{0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001}_{32\ \text{bits}}$$

Etapla 2: complemento de UM (inverte todos os bits)

$$\sim 1 = \underbrace{1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110}_{32\ \text{bits}}$$

Etapa 3: complemento de DOIS ($C1 + 1$)

$$\sim 1 + 1 = \underbrace{1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111}_{32\ \text{bits}}$$

Complemento de dois de -1 em 32 bits: 32 bits 1 (todos), ou seja:

1111 1111 1111 1111 1111 1111 1111 1111₂

Em hexadecimal: **0xFFFFFFFF**. Em decimal sem sinal: **4294967295** ($= 2^{32} - 1$).

Verificação

$$\begin{array}{r} 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001 \quad (+1) \\ + 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111 \quad (-1\ \text{em}\ C2) \\ \hline (1)\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000 \end{array}$$

O bit transbordado é descartado; o resultado em 32 bits é zero. Confirma que estamos somando o oposto.

Observação

Por que `(unsigned)(-1) == UINT_MAX` em C/C++: quando você converte -1 (`int`) para `unsigned`, não há conversão real — só reinterpretação dos mesmos bits. E os bits de -1 em C2 são 1's. Logo, `(unsigned)(-1)` é o maior `unsigned` possível:

- Em 32 bits: `UINT_MAX` = 4.294.967.295
- Em 64 bits: `UINT64_MAX` = 18.446.744.073.709.551.615

Esse truque é *idiomatico* em C: por exemplo, `string::npos` é definido como `(size_t)(-1)`.

Encerramento

Resolvemos os 11 exercícios deste apêndice. Todas as conversões foram verificadas matematicamente. Sintetizando:

- Em qualquer base b , dígitos vão de 0 a $b - 1$; valores posicionais são $b^0, b^1, b^2, \dots$
- Binário $\leftrightarrow$ octal: grupos de 3 bits. Binário $\leftrightarrow$ hex: grupos de 4 bits.
- Para outras bases, multiplique dígitos por valores posicionais.
- Complemento de 2: inverta os bits e some 1.
- Em C++: `cout << oct << n, << hex << n, << dec << n` (Cap. 23).

Com este apêndice, o livro Deitel encerra o material sobre C++. Próximos apêndices (D, E, F) cobrem outros aspectos técnicos da linguagem.